In [194…

```
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis, QuadraticDisc
from sklearn.neighbors import KNeighborsClassifier

from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.metrics import mean_squared_error
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt

from matplotlib import colors
import seaborn as sns

import scipy.stats as stats

import statsmodels.api as sm
from sklearn import tree

from sklearn.model_selection import cross_val_score
from sklearn.inspection import permutation_importance
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    balanced_accuracy_score
)
from statsmodels.stats.outliers_influence import variance_inflation_factor
import shap
```

In [195…

```
default = pd.read_csv(r"C:\\Users\\andre\\Downloads\\credit_card_defaults.csv")
print(default.isna().sum())
default.head()
```

```
ID                            0
LIMIT_BAL                     0
SEX                           0
EDUCATION                     0
MARRIAGE                      0
AGE                           0
PAY_0                         0
PAY_2                         0
PAY_3                         0
PAY_4                         0
PAY_5                         0
PAY_6                         0
BILL_AMT1                     0
BILL_AMT2                     0
BILL_AMT3                     0
BILL_AMT4                     0
BILL_AMT5                     0
BILL_AMT6                     0
PAY_AMT1                      0
PAY_AMT2                      0
PAY_AMT3                      0
PAY_AMT4                      0
PAY_AMT5                      0
PAY_AMT6                      0
default payment next month    0
dtype: int64
```

Out[195…

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	…	E
0	1	20000	2	2	1	24	2	2	-1	-1	…	
1	2	120000	2	2	2	26	-1	2	0	0	…	
2	3	90000	2	2	2	34	0	0	0	0	…	
3	4	50000	2	2	1	37	0	0	0	0	…	
4	5	50000	1	2	1	57	-1	0	-1	0	…	

5 rows × 25 columns

In [196…

```python
x = default.drop('default payment next month', axis=1)
y = default['default payment next month']

corr_matrix = x.corr().abs()
upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
to_drop = [col for col in upper.columns if any(upper[col] > 0.85)]

x_corr_filtered = x.drop(columns=to_drop)

print("\nDropped Due to High Correlation:")
print(to_drop)
```

```
Dropped Due to High Correlation:
['BILL_AMT2', 'BILL_AMT3', 'BILL_AMT4', 'BILL_AMT5', 'BILL_AMT6']
```

In [197…

```
x = default.drop(columns=['BILL_AMT2', 'BILL_AMT3', 'BILL_AMT4', 'BILL_AMT5', 'BILL
```

In [198…

```
def calculate_vif(data):
    X_const = sm.add_constant(data)
    vif = pd.DataFrame()
    vif["Feature"] = X_const.columns
    vif["VIF"] = [
        variance_inflation_factor(X_const.values, i)
        for i in range(X_const.shape[1])
    ]
    return vif

def vif_filter(data, threshold=5):
    variables = data.columns.tolist()
    while True:
        vif = calculate_vif(data[variables])
        vif = vif[vif["Feature"] != "const"]
        max_vif = vif["VIF"].max()
        if max_vif > threshold:
            drop_feature = vif.sort_values("VIF", ascending=False)["Feature"].iloc[
            variables.remove(drop_feature)
        else:
            break
    return variables

selected_vif_features = vif_filter(x_corr_filtered)
X_filtered = x_corr_filtered[selected_vif_features]

print("\nRemaining Features After VIF Filtering:")
print(selected_vif_features)
```

```
Remaining Features After VIF Filtering:
['ID', 'LIMIT_BAL', 'SEX', 'EDUCATION', 'MARRIAGE', 'AGE', 'PAY_0', 'PAY_2', 'PAY_
3', 'PAY_4', 'PAY_5', 'PAY_6', 'BILL_AMT1', 'PAY_AMT1', 'PAY_AMT2', 'PAY_AMT3', 'PAY
_AMT4', 'PAY_AMT5', 'PAY_AMT6']
```

In [199…

```
x = default[['LIMIT_BAL', 'PAY_0', 'PAY_2', 'PAY_3', 'PAY_4',
             'PAY_5', 'PAY_6', 'BILL_AMT1', 'PAY_AMT1', 'PAY_AMT2',
             'AGE', 'SEX', 'EDUCATION', 'MARRIAGE']]

y = default['default payment next month']

X_train_imp, X_test_imp, y_train_imp, y_test_imp = train_test_split(
    x, y, test_size=0.3, random_state=42
)

scaler = StandardScaler()
X_train_scaled_imp = scaler.fit_transform(X_train_imp)
X_test_scaled_imp = scaler.transform(X_test_imp)

# Convert back to DataFrame so column names are preserved
X_train_scaled_imp = pd.DataFrame(
    X_train_scaled_imp,
```

```
    columns=X_train_imp.columns,
    index=X_train_imp.index
)

X_test_scaled_imp = pd.DataFrame(
    X_test_scaled_imp,
    columns=X_test_imp.columns,
    index=X_test_imp.index
)
```

In [200…

```
# Initialize classifiers
classifiers = {
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "SVM": SVC(kernel='rbf', random_state=42),
    "LDA": LinearDiscriminantAnalysis(),
    "Logistic Regression": LogisticRegression(random_state=42),
    "kNN": KNeighborsClassifier(n_neighbors=5),
    "QDA" : QuadraticDiscriminantAnalysis(),
    "Random Forest": RandomForestClassifier(random_state=42, class_weight="balanced

}
```

In [201…

```
def calculate_feature_importance(classifiers, X_train_unscaled, X_train_scaled, y_t
    importance_results = {}

    for name, clf in classifiers.items():

        # Tree models: use unscaled data
        if name in ["Decision Tree", "Random Forest"]:
            clf.fit(X_train_unscaled, y_train)
            importance_results[name] = clf.feature_importances_

        # Coefficient-based models: use scaled data
        elif name in ["Logistic Regression", "LDA"]:
            clf.fit(X_train_scaled, y_train)
            importance_results[name] = np.abs(clf.coef_[0])

        # QDA: use permutation importance on scaled data
        elif name == "QDA":
            clf.fit(X_train_scaled, y_train)
            result = permutation_importance(
                clf,
                X_train_scaled,
                y_train,
                n_repeats=10,
                random_state=42
            )
            importance_results[name] = result.importances_mean

        # SVM with RBF kernel has no direct feature importance
        elif name == "SVM":
            importance_results[name] = None

        # kNN has no direct feature importance
        elif name == "kNN":
```

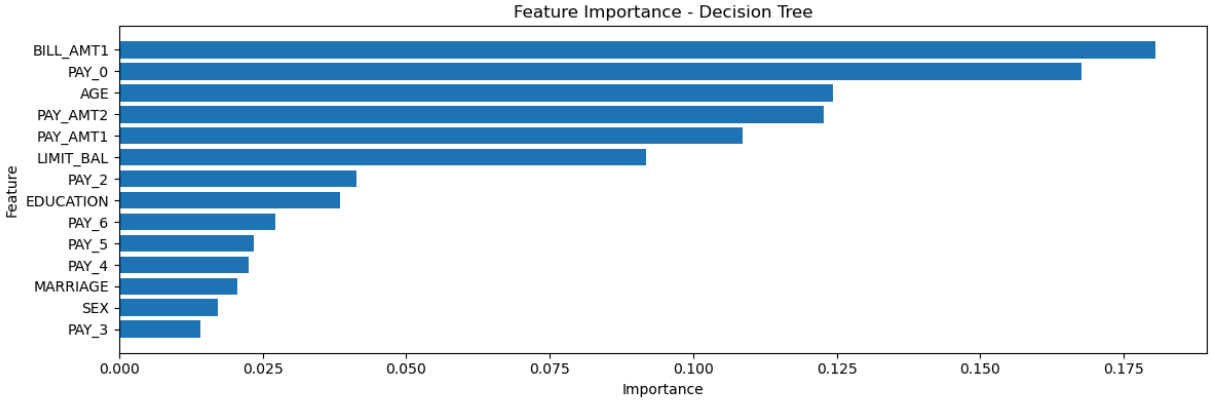
```
            importance_results[name] = None

    return importance_results
```

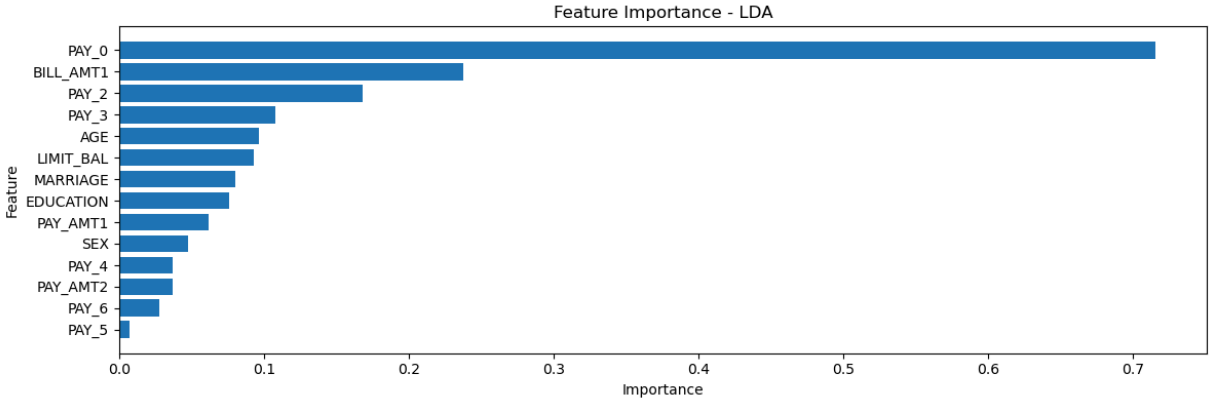
In [202…

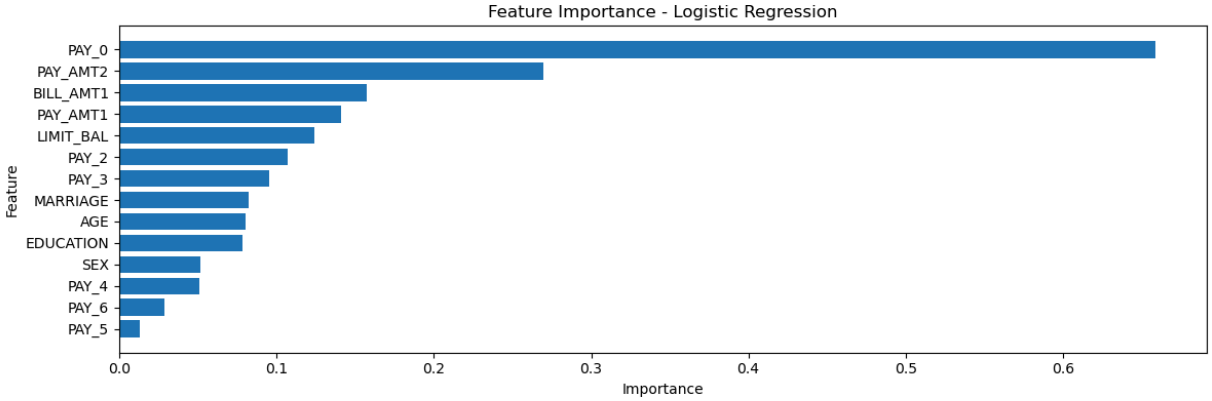
```
def plot_feature_importance(importance_values, feature_names):
    valid_items = [
        (name, imp) for name, imp in importance_values.items()
        if imp is not None
    ]

    n_models = len(valid_items)
    plt.figure(figsize=(12, 4 * n_models))

    for i, (name, importance) in enumerate(valid_items, start=1):
        sorted_idx = np.argsort(importance)
        sorted_features = np.array(feature_names)[sorted_idx]
        sorted_importance = np.array(importance)[sorted_idx]

        plt.subplot(n_models, 1, i)
        plt.barh(sorted_features, sorted_importance)
        plt.title(f"Feature Importance - {name}")
        plt.xlabel("Importance")
        plt.ylabel("Feature")

    plt.tight_layout()
    plt.show()
```

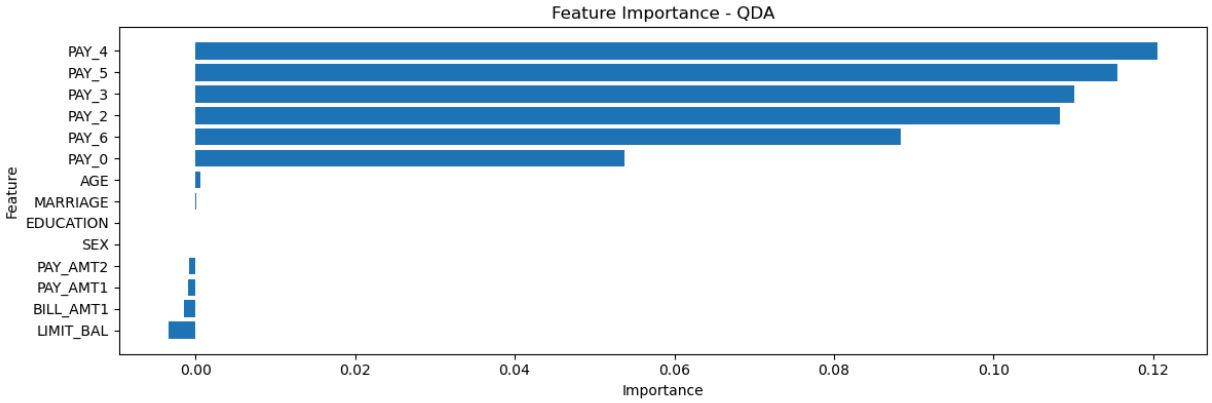
In [203…

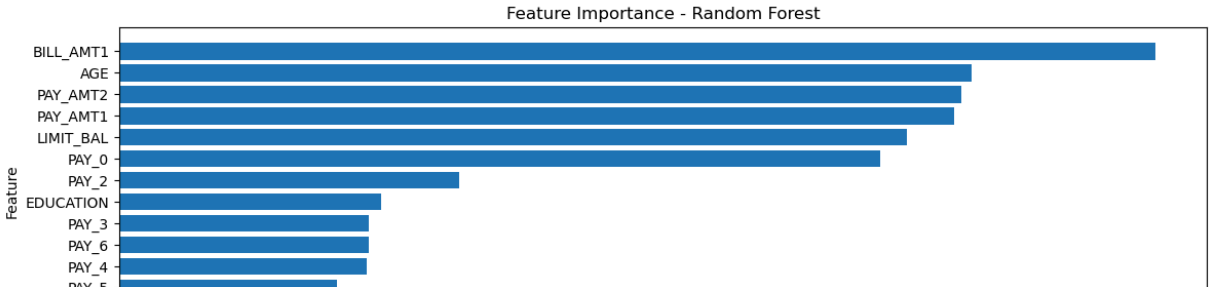
```
importance_values = calculate_feature_importance(
    classifiers,
    X_train_imp,
    X_train_scaled_imp,
    y_train_imp
)

feature_names = X_train_imp.columns
plot_feature_importance(importance_values, feature_names)
```

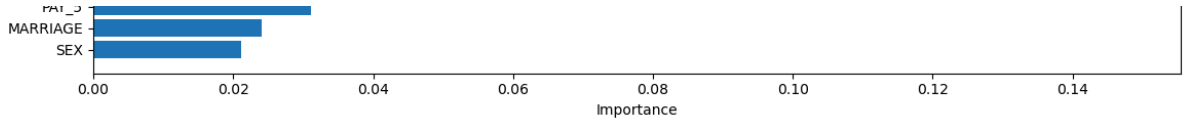

Feature Importance - Decision Tree
BILL_AMT1
PAY_0
AGE
PAY_AMT2
PAY_AMT1
LIMIT_BAL
PAY_2
EDUCATION
PAY_6
PAY_5
PAY_4
MARRIAGE
SEX
PAY_3
Feature
0.000
0.025
0.050
0.075
0.100
0.125
0.150
0.175
Importance


Feature Importance - LDA
PAY_0
BILL_AMT1
PAY_2
PAY_3
AGE
LIMIT_BAL
MARRIAGE
EDUCATION
PAY_AMT1
SEX
PAY_4
PAY_AMT2
PAY_6
PAY_5
Feature
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
Importance


Feature Importance - Logistic Regression
PAY_0
PAY_AMT2
BILL_AMT1
PAY_AMT1
LIMIT_BAL
PAY_2
PAY_3
MARRIAGE
AGE
EDUCATION
SEX
PAY_4
PAY_6
PAY_5
Feature
0.0
0.1
0.2
0.3
0.4
0.5
0.6
Importance


Feature Importance - QDA
PAY_4
PAY_5
PAY_3
PAY_2
PAY_6
PAY_0
AGE
MARRIAGE
EDUCATION
SEX
PAY_AMT2
PAY_AMT1
BILL_AMT1
LIMIT_BAL
Feature
0.00
0.02
0.04
0.06
0.08
0.10
0.12
Importance


Feature Importance - Random Forest
BILL_AMT1
AGE
PAY_AMT2
PAY_AMT1
LIMIT_BAL
PAY_0
PAY_2
EDUCATION
PAY_3
PAY_6
PAY_4
Feature



- Chose 8 features for all around performance.

In [204…
```
x = default[['BILL_AMT1', 'AGE', 'PAY_AMT2', 'PAY_AMT1', 'LIMIT_BAL', 'PAY_0', 'PAY
```

In [205…
```
X_train, X_test, y_train, y_test = train_test_split(
    x, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

ranked_features = ['BILL_AMT1', 'AGE', 'PAY_AMT2', 'PAY_AMT1', 'LIMIT_BAL', 'PAY_0'
```

In [206…
```
models = {
    "SVM": SVC(kernel="rbf", probability=True, class_weight="balanced", random_stat
    "Logistic Regression": LogisticRegression(max_iter=5000, class_weight="balanced
    "Decision Tree": DecisionTreeClassifier(max_depth=4, class_weight="balanced", r
    "Random Forest": RandomForestClassifier(
        n_estimators=200,
        class_weight="balanced",
        random_state=42,
        n_jobs=-1
    ),
    "LDA": LinearDiscriminantAnalysis(),
    "QDA": QuadraticDiscriminantAnalysis(),
    "kNN": KNeighborsClassifier(n_neighbors=5)
}

scaled_models = {"SVM", "Logistic Regression", "LDA", "QDA", "kNN"}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

In [207…
```
results = []

for model_name, model in models.items():
    X_train_eval = X_train.copy()
    X_test_eval = X_test.copy()

    if model_name in scaled_models:
        scaler = StandardScaler()
        X_train_eval = scaler.fit_transform(X_train_eval)
        X_test_eval = scaler.transform(X_test_eval)

    model.fit(X_train_eval, y_train)

    y_pred = model.predict(X_test_eval)

    # some models may not always have predict_proba in every setup
    if hasattr(model, "predict_proba"):
```

```
        y_prob = model.predict_proba(X_test_eval)[:, 1]
    else:
        y_prob = None

    cm = confusion_matrix(y_test, y_pred)
    tn, fp, fn, tp = cm.ravel()

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, zero_division=0)
    recall = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)
    specificity = tn / (tn + fp) if (tn + fp) > 0 else 0
    balanced_acc = balanced_accuracy_score(y_test, y_pred)
    roc_auc = roc_auc_score(y_test, y_prob) if y_prob is not None else np.nan

    results.append({
        "Model": model_name,
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "Specificity": specificity,
        "F1 Score": f1,
        "Balanced Accuracy": balanced_acc,
        "ROC-AUC": roc_auc,
        "Confusion Matrix": cm
    })
```

In [208…

```
from sklearn.model_selection import cross_validate, StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.metrics import make_scorer

def specificity_score(y_true, y_pred):
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    return tn / (tn + fp)

specificity = make_scorer(specificity_score)

scoring = {
    "accuracy": "accuracy",
    "precision": "precision",
    "specificity": specificity,
    "recall": "recall",
    "f1": "f1",
    "roc_auc": "roc_auc",
    "balanced_accuracy": "balanced_accuracy"
}

cv_results = []

for model_name, model in models.items():

    # Apply scaling only when necessary
    if model_name in scaled_models:
        estimator = Pipeline([
            ("scaler", StandardScaler()),
            ("model", model)
```

```
        ])
    else:
        estimator = model

    scores = cross_validate(
        estimator,
        X_train,
        y_train,
        cv=cv,
        scoring=scoring,
        n_jobs=-1
    )

    cv_results.append({
        "Model": model_name,
        "CV Accuracy": scores["test_accuracy"].mean(),
        "CV Precision": scores["test_precision"].mean(),
        "CV Specificity": scores["test_specificity"].mean(),
        "CV Recall": scores["test_recall"].mean(),
        "CV F1 Score": scores["test_f1"].mean(),
        "CV Balanced Accuracy": scores["test_balanced_accuracy"].mean(),
        "CV ROC-AUC": scores["test_roc_auc"].mean()
    })

cv_results_df = pd.DataFrame(cv_results).sort_values(
    by="CV ROC-AUC", ascending=False
)

print("Cross-Validation Results:")
print(cv_results_df)
```

```
Cross-Validation Results:
                 Model  CV Accuracy  CV Precision  CV Specificity  CV Recall  \
2        Decision Tree     0.774667      0.497358        0.837305   0.554166
0                  SVM     0.786458      0.517034        0.859130   0.530608
3        Random Forest     0.811042      0.629539        0.940720   0.354493
5                  QDA     0.782208      0.507613        0.853994   0.529480
1  Logistic Regression     0.694500      0.384073        0.712643   0.630632
4                  LDA     0.811875      0.700197        0.968166   0.261631
6                  kNN     0.797208      0.565204        0.920978   0.361461

   CV F1 Score  CV Balanced Accuracy  CV ROC-AUC
2     0.521505              0.695735    0.752522
0     0.523692              0.694869    0.750054
3     0.453488              0.647607    0.746550
5     0.518267              0.691737    0.728157
1     0.477339              0.671637    0.723298
4     0.380760              0.614899    0.719311
6     0.440898              0.641219    0.704911
```

In [209…

```
best_model_name = cv_results_df.iloc[0]["Model"]
best_model = models[best_model_name]

if best_model_name in scaled_models:
    final_model = Pipeline([
        ("scaler", StandardScaler()),
```

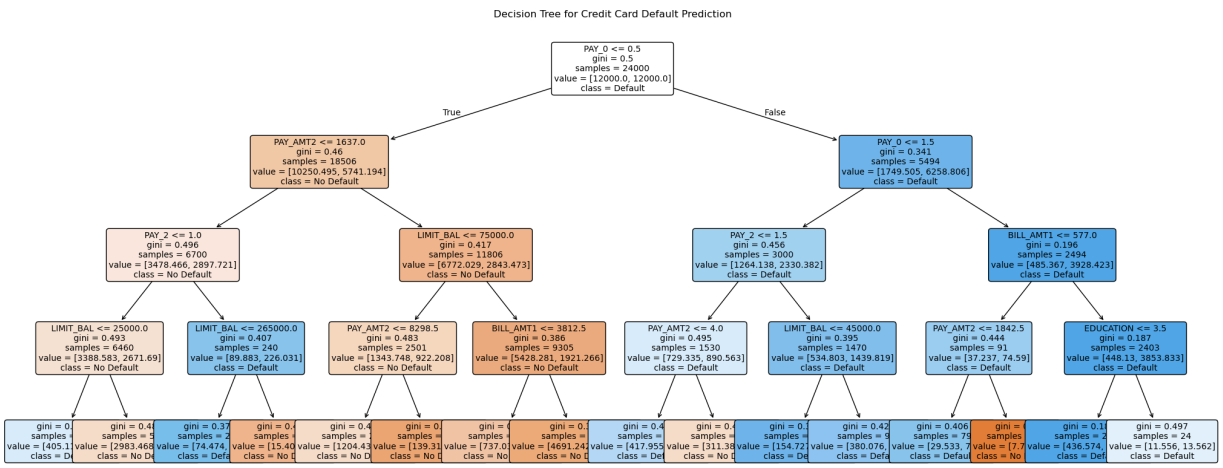
```
        ("model", best_model)
    ])
else:
    final_model = best_model

final_model.fit(X_train, y_train)

y_pred = final_model.predict(X_test)
y_prob = final_model.predict_proba(X_test)[:, 1]

from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)
print("\nBest Model:", best_model_name)
print("Confusion Matrix:\n", cm)
```

```
Best Model: Decision Tree
Confusion Matrix:
 [[3925  748]
 [ 614  713]]
```

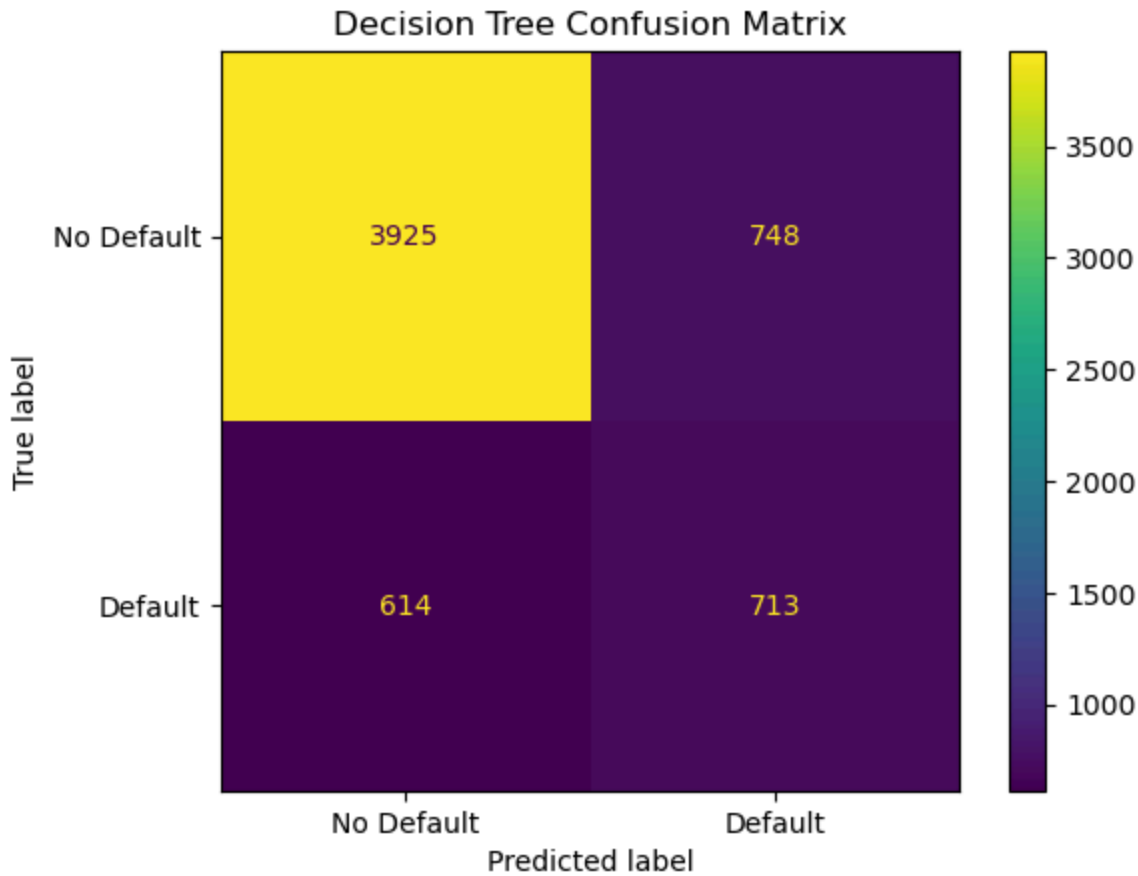
In [210…

```
# Making a decision tree for the best model
from sklearn.metrics import ConfusionMatrixDisplay
from sklearn.tree import plot_tree

tree_model = DecisionTreeClassifier(
    max_depth=4,
    class_weight="balanced",
    random_state=4598
)

tree_model.fit(X_train[ranked_features], y_train)

plt.figure(figsize=(25, 10))

plot_tree(
    tree_model,
    feature_names=ranked_features,
    class_names=["No Default", "Default"],
    filled=True,
    rounded=True,
    fontsize=10
)

plt.title("Decision Tree for Credit Card Default Prediction")
plt.show()

y_pred_tree = tree_model.predict(X_test[ranked_features])
y_prob_tree = tree_model.predict_proba(X_test[ranked_features])[:, 1]

cm_tree = confusion_matrix(y_test, y_pred_tree)

print("Decision Tree Confusion Matrix:")
print(cm_tree)

# Plot confusion matrix
```

```python
disp = ConfusionMatrixDisplay(confusion_matrix=cm_tree, display_labels=["No Default
disp.plot()
plt.title("Decision Tree Confusion Matrix")
plt.show()

tn, fp, fn, tp = cm_tree.ravel()

metrics = {
    "Accuracy": accuracy_score(y_test, y_pred_tree),
    "Precision": precision_score(y_test, y_pred_tree),
    "Recall": recall_score(y_test, y_pred_tree),
    "Specificity": tn / (tn + fp),
    "F1 Score": f1_score(y_test, y_pred_tree),
    "Balanced Accuracy": balanced_accuracy_score(y_test, y_pred_tree),
    "ROC-AUC": roc_auc_score(y_test, y_prob_tree)
}

print("\nDecision Tree Performance Metrics:")
for k, v in metrics.items():
    print(f"{k}: {v:.4f}")
```




```
Decision Tree Confusion Matrix:
[[3925  748]
 [ 614  713]]
```




```
Decision Tree Performance Metrics:
Accuracy: 0.7730
Precision: 0.4880
Recall: 0.5373
Specificity: 0.8399
F1 Score: 0.5115
Balanced Accuracy: 0.6886
ROC-AUC: 0.7398
```

In [211…

```
x_log = default[['BILL_AMT1', 'AGE', 'PAY_AMT2', 'PAY_AMT1', 'LIMIT_BAL', 'PAY_0',
y_log = default['default payment next month']

X_train_log, X_test_log, y_train_log, y_test_log = train_test_split(
    x_log, y_log, test_size=0.2, random_state=42, stratify=y_log
)

scale = StandardScaler()
x_train_log_scaled = scale.fit_transform(X_train_log)
x_test_log_scaled = scale.transform(X_test_log)

# Making logistic regression model
log_reg = LogisticRegression(max_iter=5000, class_weight="balanced", random_state=4
log_reg.fit(x_train_log_scaled, y_train_log)

y_pred_log = log_reg.predict(x_test_log_scaled)
y_prob_log = log_reg.predict_proba(x_test_log_scaled)[:, 1]
cm_log = confusion_matrix(y_test_log, y_pred_log)

print("Logistic Regression Confusion Matrix:")
```

```python
print(cm_log)

x_train_log_scaled_df = pd.DataFrame(
    x_train_log_scaled,
    columns=x_log.columns,
    index=X_train_log.index
)

X_const = sm.add_constant(x_train_log_scaled_df)
logit_model = sm.Logit(y_train_log, X_const)
result = logit_model.fit()

print(result.summary())

tn, fp, fn, tp = cm_log.ravel()

metrics = {
    "Accuracy": accuracy_score(y_test_log, y_pred_log),
    "Precision": precision_score(y_test_log, y_pred_log),
    "Recall": recall_score(y_test_log, y_pred_log),
    "Specificity": tn / (tn + fp),
    "F1 Score": f1_score(y_test_log, y_pred_log),
    "Balanced Accuracy": balanced_accuracy_score(y_test_log, y_pred_log),
    "ROC-AUC": roc_auc_score(y_test_log, y_prob_log)
}

print("\nLogistic Regression Performance Metrics:")
for k, v in metrics.items():
    print(f"{k}: {v:.4f}")
```

```
Logistic Regression Confusion Matrix:
[[3282 1391]
 [ 521  806]]
Optimization terminated successfully.
         Current function value: 0.465149
         Iterations 7
                           Logit Regression Results
==============================================================================
==
Dep. Variable:     default payment next month   No. Observations:                240
00
Model:                                  Logit   Df Residuals:                    239
91
Method:                                   MLE   Df Model:
8
Date:                        Thu, 16 Apr 2026   Pseudo R-squ.:                  0.11
98
Time:                                16:10:26   Log-Likelihood:                -1116
4.
converged:                               True   LL-Null:                       -1268
2.
Covariance Type:                    nonrobust   LLR p-value:                     0.0
00
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const         -1.4555      0.018    -79.558      0.000      -1.491      -1.420
BILL_AMT1     -0.1331      0.022     -6.150      0.000      -0.175      -0.091
AGE            0.1073      0.017      6.397      0.000       0.074       0.140
PAY_AMT2      -0.1794      0.041     -4.408      0.000      -0.259      -0.100
PAY_AMT1      -0.1492      0.034     -4.351      0.000      -0.216      -0.082
LIMIT_BAL     -0.1612      0.022     -7.334      0.000      -0.204      -0.118
PAY_0          0.6860      0.022     31.400      0.000       0.643       0.729
PAY_2          0.2019      0.021      9.758      0.000       0.161       0.242
EDUCATION     -0.0927      0.018     -5.044      0.000      -0.129      -0.057
==============================================================================

Logistic Regression Performance Metrics:
Accuracy: 0.6813
Precision: 0.3669
Recall: 0.6074
Specificity: 0.7023
F1 Score: 0.4574
Balanced Accuracy: 0.6549
ROC-AUC: 0.7021
```

- Logistic regression has lower accuracy, but features are more interpretable(coefficients).
- Decision tree has higher accuracy and can be interpreted easily with features selected.

- Feature Engineering - ignore this testing

In [212…

```
pay_cols = ['PAY_0', 'PAY_2', 'PAY_3', 'PAY_4', 'PAY_5', 'PAY_6']
bill_cols = ['BILL_AMT1', 'BILL_AMT2', 'BILL_AMT3',
```

```
             'BILL_AMT4', 'BILL_AMT5', 'BILL_AMT6']
pay_amt_cols = ['PAY_AMT1', 'PAY_AMT2', 'PAY_AMT3',
                'PAY_AMT4', 'PAY_AMT5', 'PAY_AMT6']

default['AVG_PAY_STATUS'] = default[pay_cols].mean(axis=1)
default['MAX_PAY_STATUS'] = default[pay_cols].max(axis=1)
default['LATE_PAYMENT_COUNT'] = (default[pay_cols] > 0).sum(axis=1)

# Financial features
default['AVG_BILL_AMT'] = default[bill_cols].mean(axis=1)
default['AVG_PAY_AMT'] = default[pay_amt_cols].mean(axis=1)

# Ratios
default['CREDIT_UTILIZATION'] = default['AVG_BILL_AMT'] / default['LIMIT_BAL']
default['PAYMENT_RATIO'] = (
    default[pay_amt_cols].sum(axis=1) /
    (default[bill_cols].sum(axis=1) + 1)
)

# Replace infinities and NaNs
default.replace([np.inf, -np.inf], np.nan, inplace=True)
default.fillna(0, inplace=True)
```

In [213…

```
features = [
    'AVG_PAY_STATUS',
    'MAX_PAY_STATUS',
    'LATE_PAYMENT_COUNT',
    'AVG_BILL_AMT',
    'AVG_PAY_AMT',
    'CREDIT_UTILIZATION',
    'PAYMENT_RATIO',
    'AGE',
    'LIMIT_BAL'
]

X_eng = default[features]
y_eng = default['default payment next month']
```

In [214…

```
X_train_eng, X_test_eng, y_train_eng, y_test_eng = train_test_split(
    X_eng, y_eng,
    test_size=0.20,
    random_state=42,
    stratify=y_eng
)
```

In [215…

```
models1 = {
    "SVM": SVC(kernel="rbf", probability=True,
               class_weight="balanced", random_state=42),
    "Logistic Regression": LogisticRegression(
        max_iter=5000, class_weight="balanced", random_state=42),
    "Decision Tree": DecisionTreeClassifier(
        max_depth=4, class_weight="balanced", random_state=42),
    "Random Forest": RandomForestClassifier(
        n_estimators=150, class_weight="balanced",
        random_state=42, n_jobs=-1),
```

```
    "LDA": LinearDiscriminantAnalysis(),
    "QDA": QuadraticDiscriminantAnalysis(),
    "kNN": KNeighborsClassifier(n_neighbors=5)
}

scaled_models1 = {"SVM", "Logistic Regression", "LDA", "QDA", "kNN"}
```

In [216…

```
results = []

for model_name, model in models1.items():
    X_train_eval_eng = X_train_eng.copy()
    X_test_eval_eng = X_test_eng.copy()

    # Scale when necessary
    if model_name in scaled_models1:
        scaler = StandardScaler()
        X_train_eval_eng = scaler.fit_transform(X_train_eval_eng)
        X_test_eval_eng = scaler.transform(X_test_eval_eng)

    # Fit model
    model.fit(X_train_eval_eng, y_train_eng)
    y_pred = model.predict(X_test_eval_eng)

    # Probabilities for ROC-AUC
    y_prob = model.predict_proba(X_test_eval_eng)[:, 1]

    # Confusion matrix
    cm = confusion_matrix(y_test_eng, y_pred)
    tn, fp, fn, tp = cm.ravel()

    results.append({
        "Model": model_name,
        "Accuracy": accuracy_score(y_test_eng, y_pred),
        "Precision": precision_score(y_test, y_pred, zero_division=0),
        "Recall": recall_score(y_test, y_pred, zero_division=0),
        "Specificity": tn / (tn + fp),
        "F1 Score": f1_score(y_test, y_pred, zero_division=0),
        "Balanced Accuracy": balanced_accuracy_score(y_test, y_pred),
        "ROC-AUC": roc_auc_score(y_test, y_prob),
        "Confusion Matrix": cm
    })

# Convert results to DataFrame
results_df = pd.DataFrame(results).sort_values(
    by=["ROC-AUC", "Recall", "F1 Score"],
    ascending=False
)
```

In [217…

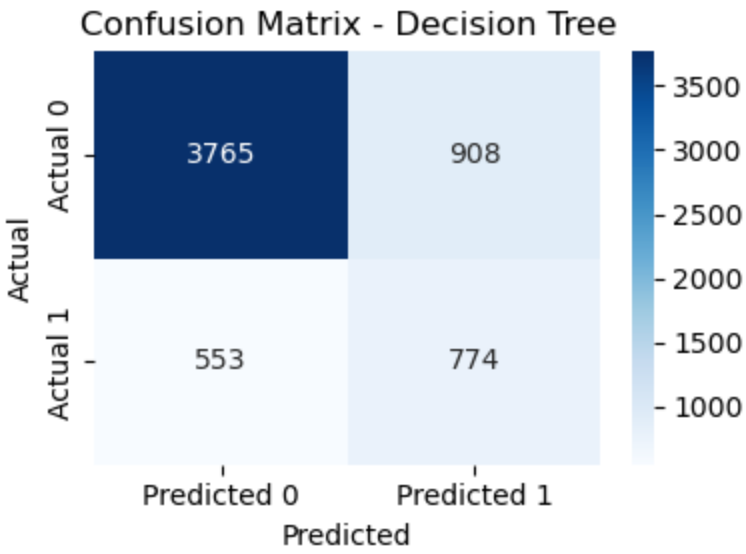
```
print("\nModel Performance Comparison:")
print(results_df[[
    "Model", "Accuracy", "Precision", "Recall",
    "Specificity", "F1 Score",
    "Balanced Accuracy", "ROC-AUC"
]])
```

```
Model Performance Comparison:
                 Model  Accuracy  Precision    Recall  Specificity  F1 Score  \
2        Decision Tree  0.756500   0.460166  0.583271     0.805692  0.514457
1  Logistic Regression  0.737500   0.433690  0.611153     0.773379  0.507351
5                  QDA  0.518667   0.292806  0.831198     0.429917  0.433059
4                  LDA  0.805667   0.615165  0.324039     0.942435  0.424482
0                  SVM  0.733500   0.428421  0.613414     0.767601  0.504493
3        Random Forest  0.796000   0.574315  0.299925     0.936871  0.394059
6                  kNN  0.777167   0.494226  0.322532     0.906270  0.390333

   Balanced Accuracy   ROC-AUC
2           0.694481  0.747765
1           0.692266  0.741877
5           0.630557  0.740240
4           0.633237  0.738982
0           0.690507  0.737675
3           0.618398  0.731562
6           0.614401  0.694303
```

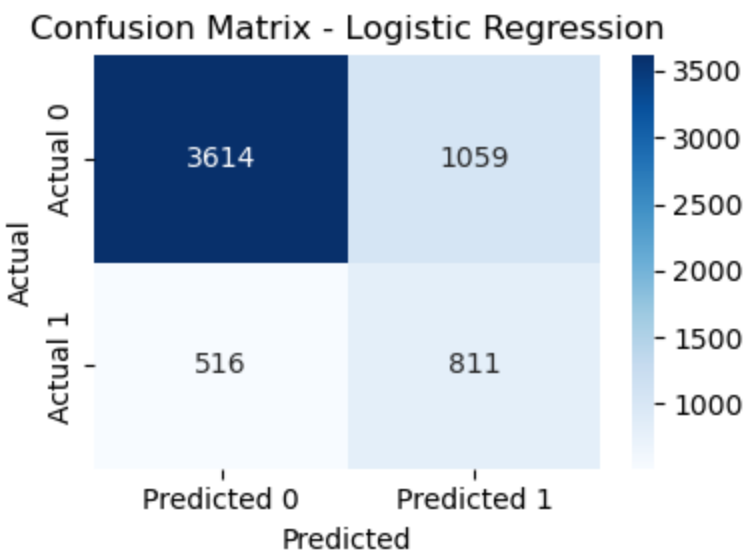
In [218…

```python
for _, row in results_df.iterrows():
    print("\n" + "=" * 50)
    print(f"Model: {row['Model']}")
    print("Confusion Matrix:")
    print(row["Confusion Matrix"])

    plt.figure(figsize=(4, 3))
    sns.heatmap(row["Confusion Matrix"], annot=True, fmt="d",
                cmap="Blues",
                xticklabels=["Predicted 0", "Predicted 1"],
                yticklabels=["Actual 0", "Actual 1"])
    plt.title(f"Confusion Matrix - {row['Model']}")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.tight_layout()
    plt.show()
```

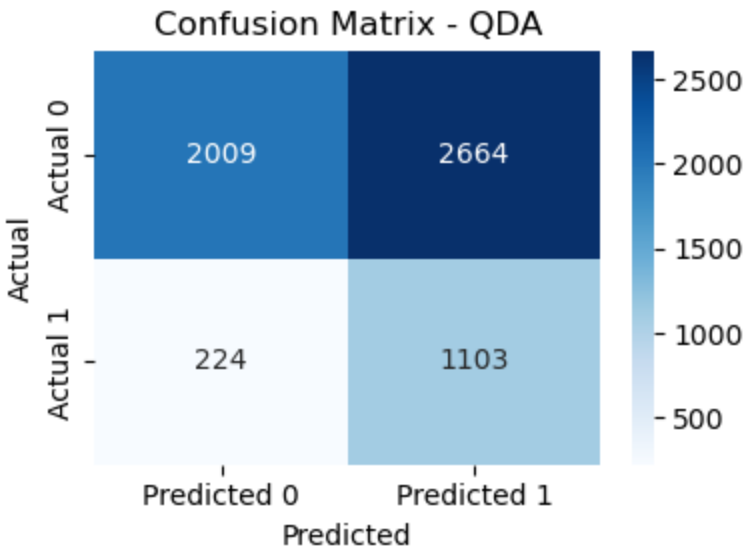
```
==================================================
Model: Decision Tree
Confusion Matrix:
[[3765  908]
 [ 553  774]]
```



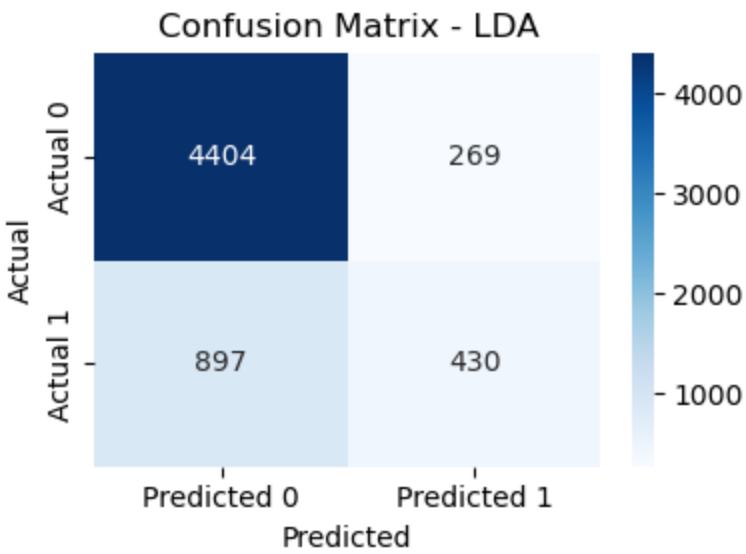

```
==================================================
Model: Logistic Regression
Confusion Matrix:
[[3614 1059]
 [ 516  811]]
```



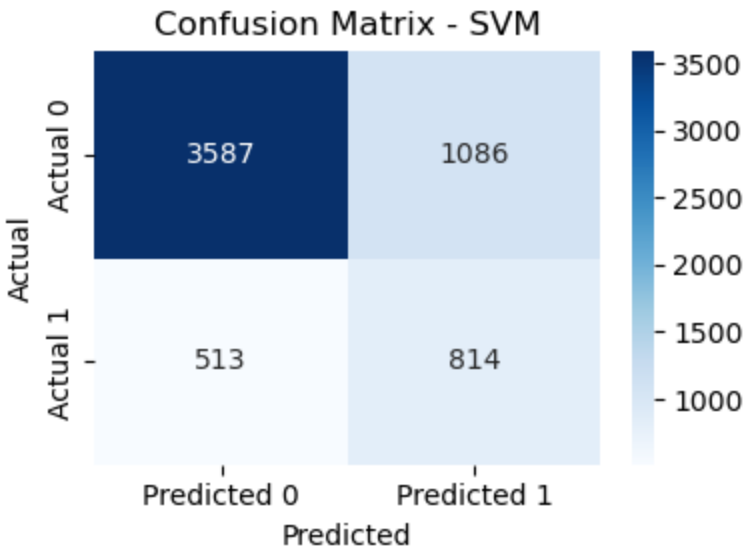

```
==================================================
Model: QDA
Confusion Matrix:
[[2009 2664]
 [ 224 1103]]
```



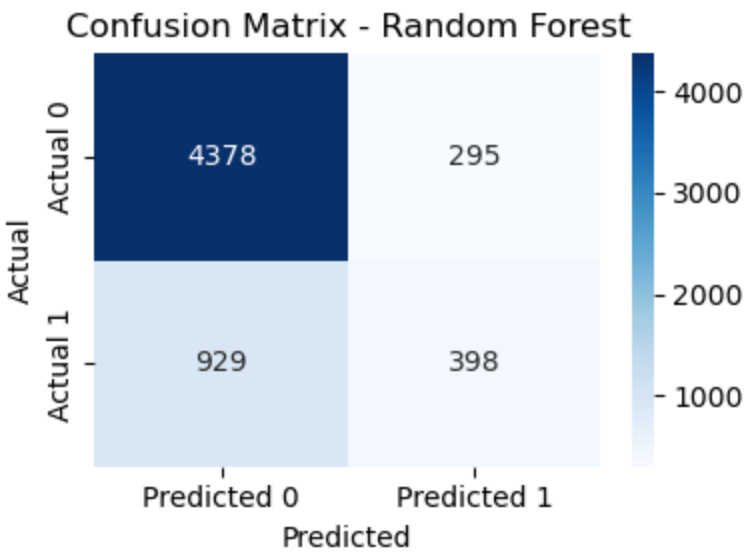

```
==================================================
Model: LDA
Confusion Matrix:
[[4404  269]
 [ 897  430]]
```



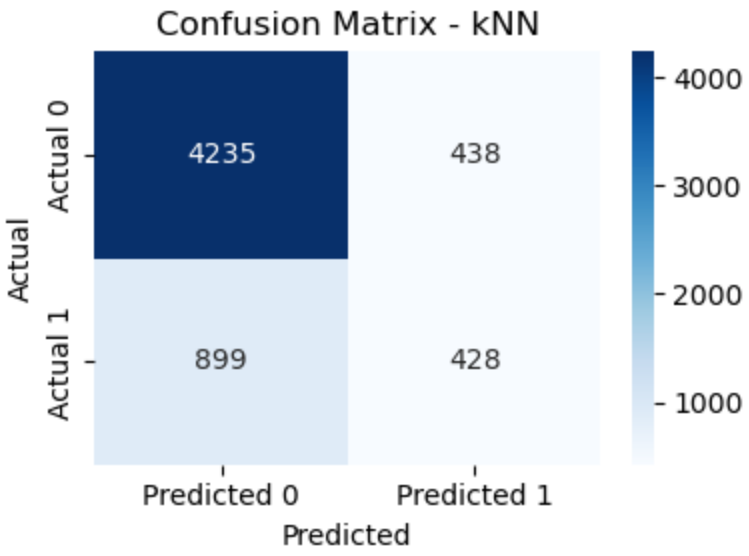

```
==================================================
Model: SVM
Confusion Matrix:
[[3587 1086]
 [ 513  814]]
```




```
==================================================
Model: Random Forest
Confusion Matrix:
[[4378  295]
 [ 929  398]]
```




```
==================================================
Model: kNN
Confusion Matrix:
[[4235  438]
 [ 899  428]]
```


Confusion Matrix - kNN
Actual 0
Actual 1
Actual
4235
438
899
428
Predicted 0
Predicted 1
Predicted
4000
3000
2000
1000